Notas GeoPython — Día 1

> Resumen de conceptos vistos durante la introducción a Python en Google Colab

---

1. CPU vs GPU vs TPU en Google Colab

Procesador	Núcleos	Ideal para	En Colab
CPU	Pocos, muy potentes	Cualquier tarea general	Por defecto
GPU	Miles, en paralelo	Redes neuronales, IA	NVIDIA T4, A100...
TPU	Especializado en tensores	Modelos grandes con TensorFlow/JAX	Disponible en Colab

Para un curso de **GeoPython** con Python básico se trabaja casi todo en **CPU**, es más que suficiente.

Para cambiar el tipo de acelerador: `Entorno de ejecución → Cambiar tipo de entorno de ejecución`

---

2. Google Colab — Lo esencial

Atajos de teclado

- `Shift + Enter` — ejecuta la celda y pasa a la siguiente
- `Ctrl + Enter` — ejecuta la celda y se queda en ella

Cosas a tener en cuenta

- Las variables persisten entre celdas mientras el entorno esté activo
- Si llevas un rato sin usarlo, **el entorno se reinicia** y pierdes las variables (pero no el código)
- Para instalar librerías: `!pip install nombre_libreria`
- Los archivos subidos se pierden al reiniciar → conectar Google Drive para persistencia

Easter eggs 🐕 🐱🦀

En `Herramientas → Configuración → Modos` puedes activar:

- **Nivel de poder** ⚡

- **Modo Corgi** 🐶
- **Modo Kitty** 🐱
- **Modo Cangrejo** 🦀

---

3. Funciones — Casos especiales

Guarda defensiva para evitar división por cero

```python
def normalizar(valores):
    """Normaliza una lista al rango [0, 1]."""
    vmin, vmax = min(valores), max(valores)
    if vmin == vmax:
        return [0.0] * len(valores)  # evita división por cero
    return [(v - vmin) / (vmax - vmin) for v in valores]
```

¿Por qué `[0.0] * len(valores)`?

- Si todos los valores son iguales (ej: `[5, 5, 5]`), entonces `vmax - vmin = 0` → división por cero 💥
- En ese caso no hay variación que normalizar → se devuelve una lista de ceros del mismo tamaño

---

4. Clases en Python

Tipos de métodos y decoradores

```python
```

```python
class Perro:
    especie = 'Canis lupus familiaris'  # atributo de clase, compartido
    _registro = []

    def __init__(self, nombre, raza, edad):
        self.nombre = nombre
        self.raza = raza
        self.edad = edad           # llama al setter si existe
        Perro._registro.append(self)

    def __repr__(self):
        return f"Perro('{self.nombre}', '{self.raza}', {self.edad})"
```

`@classmethod` — recibe `cls`

Accede a la clase en sí, no a una instancia concreta. Se usa para atributos de clase o constructores alternativos.

```python
@classmethod
def cuantos_hay(cls):
    return f'Hay {len(cls._registro)} perro(s) registrado(s)'

@classmethod
def desde_string(cls, texto):
    """Constructor alternativo desde un string 'nombre,raza,edad'"""
    nombre, raza, edad = texto.split(',')
    return cls(nombre.strip(), raza.strip(), int(edad.strip()))

# Uso:
Perro.cuantos_hay()
Perro.desde_string('Rex, Labrador, 4')
```

`@staticmethod` — sin `self` ni `cls`

Función normal que vive dentro de la clase por organización. No necesita saber nada de la clase ni de la instancia.

```python
```

```python
@staticmethod
def es_nombre_valido(nombre):
    return bool(nombre) and nombre.replace(' ', '').isalpha()

# Uso:
Perro.es_nombre_valido('Rex')   # no necesita ningún perro creado
```

Resumen de decoradores

Decorador	Primer parámetro	Accede a...	Se llama desde...
Método normal	`self`	la instancia	un objeto
`@classmethod`	`cls`	la clase	la clase o un objeto
`@staticmethod`	ninguno	nada	la clase o un objeto
`@property`	`self`	la instancia	como atributo (sin `()`)

---

5. `@property` y el guion bajo `_`

¿Para qué sirve `@property`?

Permite usar un método como si fuera un atributo (sin paréntesis) y controlar el acceso.

```python
cop.edad      # ✅ sin ()
cop.edad()    # ❌ error
```

Sin property no puedes controlar qué se guarda:

```python
cop.edad = -100   # Python lo acepta sin quejarse 😬
cop.edad = "hola" # también 😬
```

Con property + setter puedes validar:

```python
```

```
cop.edad = -100   # ❌ ValueError
cop.edad = "hola" # ❌ ValueError
```

El guion bajo `_edad`

Es una convención que significa "atributo interno, no accedas directamente desde fuera".

```python
@property
def edad(self):
    return self._edad        # getter: devuelve el valor interno

@edad.setter
def edad(self, valor):
    if not isinstance(valor, int) or valor < 0:
        raise ValueError(f'La edad debe ser un entero no negativo, no {valor!r}')
    self._edad = valor       # setter: valida y guarda en _edad
```

¿Por qué necesitamos `_edad` y no `edad` dentro del setter?

Si usaras `self.edad = valor` dentro del setter → bucle infinito (se llama a sí mismo eternamente) 💥

La metáfora de la caja fuerte:

```
cop.edad = -5
      ↓
  setter (comprueba si es válido)
      ↓
  ❌ ValueError, no entra

cop._edad = -5
      ↓
  vas directo a la caja fuerte saltándote la puerta
      ↓
  ✅ se guarda sin validar (malo)
```

---

6. Cálculo de edad — El problema del cumpleaños

❌ Versión incorrecta (solo año)

```python
@property
def edad(self):
    return date.today().year - self.anio_nacimiento
```

Problema: si naciste en junio de 2000 y estamos en abril de 2026, devuelve `26` pero tienes `25`. Aún no has cumpleaños.

✅ Versión correcta (con fecha completa)

```python
from datetime import date

@property
def edad(self):
    hoy = date.today()
    anios = hoy.year - self.fecha_nacimiento.year

    # Si aún no ha llegado el cumpleaños este año, restamos 1
    if (hoy.month, hoy.day) < (self.fecha_nacimiento.month, self.fecha_nacimiento.da
        anios -= 1

    return anios
```

El truco de comparar tuplas `(mes, día)` funciona porque Python las compara elemento a elemento.

✅ Versión con Arrow (más limpia)

```python
import arrow

@property
def edad(self):
    nacimiento = arrow.get(self.fecha_nacimiento)
    return (arrow.now() - nacimiento).years  # maneja el cumpleaños automáticamente
```

Arrow ya gestiona internamente el caso del cumpleaños no llegado, sin necesidad de comparar tuplas.

> **Nota:** En GeoPython es más común ver `datetime` nativo o `pandas` para fechas. Arrow es más habitual en proyectos web o análisis general.

Notas generadas durante el Día 1 del curso de GeoPython